

StayNexApp

NexCore · casarural-v2 · Debacu

ANÁLISIS ARQUITECTÓNICO COMPLETO

Documento de referencia técnica · Arquitecto Senior SaaS

Versión 1.0 · Abril 2026

1. Análisis Funcional Global

El sistema StayNexApp se compone de tres bases de datos Supabase independientes con responsabilidades claramente separadas. Esta separación es correcta y debe mantenerse en toda la implementación.

1.1 Arquitectura de Tres Capas

Sistema	Supabase	Responsabilidad
NexCore	SUPABASE-B	Orquestador SaaS. Clientes, planes, suscripciones, provisioning, Stripe Billing, control de acceso a módulos.
casarural-v2	SUPABASE-A	Producto operativo del cliente. Properties, unidades, reservas, pagos, Stripe Connect, CRM, automatizaciones.
Debacu	SUPABASE-C	API independiente de riesgo. Screening, scoring, señales, incidencias. Consumible por terceros (PMS, hotels).

1.2 Módulos Funcionales de NexCore

Área	Módulos
CRM / Clientes	Alta comercial, estado del cliente (LEAD/ACTIVE/SUSPENDED/CANCELLED), contacto principal, notas internas
Suscripciones	Catálogo de planes, billing mensual/anual, setup fee, trial, cambio de plan, cancelación
Facturación SaaS	Stripe Billing, invoices, impagos, suspensión automática
Provisioning	Alta en casarural-v2, alta en Debacu API, sync de límites y flags, estado de instancias
Integraciones	Gestión de Stripe Connect accounts, permisos Debacu por plan, webhooks entrantes
Soporte / Onboarding	Estado de wizard, incidencias de cliente, seguimiento técnico
Configuración	Catálogo de planes editable, feature flags, límites por plan
Reporting	Métricas SaaS: clientes activos, MRR, churn, uso por módulo
Auditoría	Audit log de todas las acciones críticas: quién, qué, cuándo

1.3 Funcionalidades Implícitas (No Diseñadas Pero Obligatorias)

Las siguientes funcionalidades no aparecen explícitamente en los documentos fuente pero son necesarias para que el sistema funcione en producción:

- Webhook de Stripe Billing en NexCore — detectar impagos SaaS y suspender automáticamente
- Mecanismo de sync NexCore → casarural-v2 — protocolo HTTP firmado con secret compartido
- Gestión de errores de provisioning — máquina de estados con pasos atómicos y retry granular
- Rotación y revocación de acceso al suspender un cliente — bloqueo propagado a casarural-v2
- Notificaciones internas al staff cuando hay eventos críticos (impago, suspensión, error de provisioning)

- Idempotencia de provisioning — si el job falla y reintenta, no debe crear duplicados
- Referencia cruzada SUPABASE-B ↔ SUPABASE-A — guardar el supabase_user_id externo del cliente provisionado

2. Modelo de Datos (SQL Real — PostgreSQL)

Todas las tablas siguen estas reglas: claves primarias UUID, timestamps en todas las tablas, RLS habilitado, lógica de negocio solo en Edge Functions (nunca en triggers ni funciones PL/pgSQL de aplicación).

2.1 NexCore — SUPABASE-B

Tabla: `saas_clients`

```
CREATE TABLE saas_clients (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  legal_name  text NOT NULL,
  trade_name  text,
  tax_id      text,
  contact_name text NOT NULL,
  contact_email text NOT NULL UNIQUE,
  contact_phone text,
  status      text NOT NULL DEFAULT 'ACTIVE'
              CHECK (status IN ('LEAD', 'ACTIVE', 'SUSPENDED', 'CANCELLED')),
  notes       text,
  created_at  timestampz NOT NULL DEFAULT now(),
  updated_at  timestampz NOT NULL DEFAULT now()
);
```

Tabla: `saas_plans`

```
CREATE TABLE saas_plans (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  code        text NOT NULL UNIQUE, -- BASIC, PRO, PREMIUM, ENTERPRISE
  name        text NOT NULL,
  monthly_price_cents integer NOT NULL,
  yearly_price_cents integer NOT NULL,
  setup_fee_cents integer NOT NULL DEFAULT 0,
  max_properties integer NOT NULL DEFAULT 1,
  max_units    integer NOT NULL DEFAULT 5,
  max_users    integer NOT NULL DEFAULT 3,
  max_domains  integer NOT NULL DEFAULT 1,
  debacu_enabled boolean NOT NULL DEFAULT false,
  crm_enabled  boolean NOT NULL DEFAULT true,
  dynamic_pricing_enabled boolean NOT NULL DEFAULT false,
  advanced_reporting boolean NOT NULL DEFAULT false,
  multi_property_enabled boolean NOT NULL DEFAULT false,
  api_calls_per_day integer NOT NULL DEFAULT 100,
  is_active    boolean NOT NULL DEFAULT true,
  created_at  timestampz NOT NULL DEFAULT now()
);
```

Tabla: `saas_subscriptions`

```
CREATE TABLE saas_subscriptions (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id   uuid NOT NULL REFERENCES saas_clients(id),
  plan_id     uuid NOT NULL REFERENCES saas_plans(id),
  billing_cycle text NOT NULL CHECK (billing_cycle IN ('MONTHLY', 'YEARLY')),
  status      text NOT NULL DEFAULT 'ACTIVE'
              CHECK (status IN
('TRIAL', 'ACTIVE', 'PAST_DUE', 'SUSPENDED', 'CANCELLED')),
  stripe_customer_id text,
  stripe_subscription_id text,
  current_period_start timestampz,
  current_period_end timestampz,
  trial_ends_at timestampz,
  cancel_at timestampz,
  cancelled_at timestampz,
  created_at timestampz NOT NULL DEFAULT now(),
  updated_at timestampz NOT NULL DEFAULT now()
);
-- Solo una suscripción activa por cliente
```

```
CREATE UNIQUE INDEX idx_saas_subscriptions_active_client
ON saas_subscriptions(client_id)
WHERE status NOT IN ('CANCELLED');
```

Tabla: client_product_instances

```
CREATE TABLE client_product_instances (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id         uuid NOT NULL REFERENCES saas_clients(id),
  product_code      text NOT NULL, -- CASARURAL V2, DEBACU_API
  status            text NOT NULL DEFAULT 'PROVISIONING'
  CHECK (status IN ('PROVISIONING','ACTIVE','SUSPENDED','ERROR')),
  supabase_project_ref text,
  external_tenant_id text,          -- UUID del tenant en casarural-v2
  external_property_id text,        -- UUID de la property base en casarural-v2
  supabase_user_id  text,          -- UUID del user en Auth de SUPABASE-A
  domain            text,
  onboarding_status text NOT NULL DEFAULT 'PENDING'
  CHECK (onboarding_status IN ('PENDING','IN_PROGRESS','COMPLETED')),
  last_synced_at    timestampz,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now(),
  UNIQUE (client_id, product_code)
);
```

Tabla: client_stripe_accounts

```
CREATE TABLE client_stripe_accounts (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id         uuid NOT NULL REFERENCES saas_clients(id),
  product_code      text NOT NULL DEFAULT 'CASARURAL_V2',
  stripe_account_id text NOT NULL UNIQUE,
  stripe_connect_type text NOT NULL DEFAULT 'express',
  onboarding_completed boolean NOT NULL DEFAULT false,
  charges_enabled   boolean NOT NULL DEFAULT false,
  payouts_enabled   boolean NOT NULL DEFAULT false,
  mode              text NOT NULL DEFAULT 'test' CHECK (mode IN ('test','live')),
  status_checked_at timestampz,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now()
);
```

Tabla: provisioning_jobs

```
CREATE TABLE provisioning_jobs (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id         uuid NOT NULL REFERENCES saas_clients(id),
  target_product    text NOT NULL,
  action            text NOT NULL,
  -- CREATE_INSTANCE | SYNC_LIMITS | SUSPEND | REACTIVATE | SYNC_STRIPE_CONNECT
  payload           jsonb NOT NULL DEFAULT '{}',
  steps             jsonb NOT NULL DEFAULT '{}', -- estado atómico por paso
  status            text NOT NULL DEFAULT 'PENDING'
  CHECK (status IN ('PENDING','RUNNING','DONE','FAILED')),
  attempts          integer NOT NULL DEFAULT 0,
  error_message     text,
  created_at        timestampz NOT NULL DEFAULT now(),
  updated_at        timestampz NOT NULL DEFAULT now()
);
CREATE INDEX idx_provisioning_jobs_pending
ON provisioning_jobs(status) WHERE status IN ('PENDING','RUNNING');
```

Tabla: saas_invoices

```
CREATE TABLE saas_invoices (
  id                uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id         uuid NOT NULL REFERENCES saas_clients(id),
  subscription_id   uuid REFERENCES saas_subscriptions(id),
  stripe_invoice_id text UNIQUE,
  amount_cents      integer NOT NULL,
  currency          text NOT NULL DEFAULT 'eur',
  status            text NOT NULL
  CHECK (status IN ('DRAFT','OPEN','PAID','VOID','UNCOLLECTIBLE')),
  period_start      timestampz,
```

```

period_end      timestamptz,
paid_at         timestamptz,
due_date        timestamptz,
created_at      timestamptz NOT NULL DEFAULT now()
);

```

Tabla: audit_logs (obligatoria, no opcional)

```

CREATE TABLE audit_logs (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  client_id   uuid REFERENCES saas_clients(id),
  actor       text NOT NULL,    -- email del staff que ejecutó
  action      text NOT NULL,    -- CREATE_CLIENT | CHANGE_PLAN | SUSPEND | etc.
  entity      text NOT NULL,    -- nombre de la tabla afectada
  entity_id   uuid,
  old_data    jsonb,
  new_data    jsonb,
  ip          text,
  created_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX idx_audit_logs_client ON audit_logs(client_id);
CREATE INDEX idx_audit_logs_created ON audit_logs(created_at DESC);

```

2.2 casarural-v2 — SUPABASE-A (campos añadidos por NexCore)

Estos campos se añaden a la tabla properties existente. Son copia operativa sincronizada desde NexCore. La fuente de verdad permanece en NexCore.

```

ALTER TABLE properties ADD COLUMN IF NOT EXISTS
  platform_client_id  uuid,      -- referencia a saas_clients.id en NexCore
  plan_code           text,      -- BASIC | PRO | PREMIUM | ENTERPRISE
  max_units           integer,
  max_domains         integer,
  max_users           integer,
  debacu_enabled      boolean NOT NULL DEFAULT false,
  crm_enabled         boolean NOT NULL DEFAULT true,
  dynamic_pricing_enabled boolean NOT NULL DEFAULT false,
  subscription_status text NOT NULL DEFAULT 'ACTIVE',
  platform_config_version integer NOT NULL DEFAULT 0,
  platform_last_synced_at timestampz;

```

REGLA CRÍTICA — Copia operativa vs Fuente de verdad

Los campos platform_* en casarural-v2 son copia operativa. Nunca se modifican directamente desde casarural-v2. Solo NexCore los actualiza a través de la función receive-platform-config-sync. Si hay discrepancia, NexCore prevalece siempre.

3. Relaciones Entre Sistemas

3.1 Diagrama de Control

La jerarquía de control es estricta y unidireccional en cuanto a decisiones de negocio:

```
NexCore (SUPABASE-B) — FUENTE DE VERDAD GLOBAL
├─ Decide: plan, límites, features habilitadas, estado cliente
├─ Controla: Stripe Billing (cuotas SaaS)
├─ Crea/gestiona: Stripe Connect accounts
├─ Provisiona → casarural-v2 y Debacu

casarural-v2 (SUPABASE-A) — EJECUTOR OPERATIVO
├─ Recibe: configuración desde NexCore vía sync
├─ Opera: reservas, pagos Connect, CRM, unidades
├─ Valida: contra copia local de límites
├─ NO decide: lógica de planes, precios SaaS, permisos de módulos

Debacu (SUPABASE-C) — API INDEPENDIENTE
├─ Expone: POST /v1/evaluate, POST /v1/incidents
├─ Produce: score, señales, recomendación
├─ Es consumido por: casarural-v2, PMS externos, futuros partners
```

3.2 Tabla de Fuentes de Verdad

Dato	Fuente de Verdad	Copia Operativa
Plan contratado	NexCore (saas_plans)	casarural-v2 (plan_code)
Límites (max_units, etc.)	NexCore (saas_plans)	casarural-v2 (max_units, ...)
Estado suscripción	NexCore + Stripe	casarural-v2 (subscription_status)
Stripe Connect account	NexCore (client_stripe_accounts)	casarural-v2 (stripe_account_id)
flags de módulos (Debacu...)	NexCore (saas_plans)	casarural-v2 (debacu_enabled, ...)
Reservas	casarural-v2	—
Pagos de reservas	casarural-v2 + Stripe	—
Score de riesgo	Debacu	casarural-v2 (snapshot en reserva)
Incidencias de huéspedes	Debacu	—
Facturas SaaS	NexCore + Stripe Billing	—

3.3 Stripe — Dos Contextos Separados

Contexto	Stripe Billing (plataforma)	Stripe Connect (reservas)
Vive en	NexCore (SUPABASE-B)	casarural-v2 (SUPABASE-A)
Para qué	Cobrar cuotas SaaS al cliente	Cobrar reservas de huéspedes
Cuenta Stripe	Cuenta principal StayNex	Connected Account (acct_xxx) del propietario
Naming en BD	saas_stripe_customer_id	stripe_account_id

	saas_stripe_subscription_id	stripe_payment_intent
--	-----------------------------	-----------------------

NAMING CRÍTICO — No mezclar contextos Stripe

Nunca usar `stripe_customer_id` sin prefijo. En NexCore todos los campos Stripe llevan prefijo `saas_` para evitar confusión con los campos Stripe Connect de `casarural-v2`.

4. Modelo Multi-Tenant

4.1 NexCore — Sin Multi-Tenant de Clientes

NexCore es un sistema de un único tenant: el equipo StayNex (staff interno). No hay aislamiento entre clientes aquí porque los clientes finales no acceden a NexCore. Lo que sí necesita es control de roles de staff y auditoría completa.

4.2 casarural-v2 — Multi-Tenant Real

Cada property es un tenant. El aislamiento debe ser absoluto: un propietario de la property A nunca puede ver datos de la property B.

Jerarquía de aislamiento:

```
Supabase Auth user
├── property_users (user_id ↔ property_id, active)
│   └── property (el tenant)
│       ├── unidades
│       ├── reservas
│       ├── pagos
│       ├── bloqueos
│       ├── reservation_holds
│       └── configuración
```

4.3 Riesgos si se Implementa Mal

Riesgo	Consecuencia
RLS deshabilitado en alguna tabla	Cualquier usuario autenticado puede leer todos los datos de todos los clientes con un SELECT simple.
property_id en URL sin validación backend	Un usuario cambia el UUID en la URL y accede a reservas o pagos de otra property.
Edge Functions sin verificar ownership	Un usuario puede llamar a create-connected-account con un property_id ajeno y crear cuentas Stripe sobre propiedades que no le pertenecen.
Service role sin audit log	Las funciones de provisioning de NexCore que usan service role no dejan rastro de qué se creó, cuándo y por quién.
Validación de plan solo en frontend	Un usuario puede inspeccionar el código, saltarse la validación y crear más unidades de las permitidas por su plan.

5. Políticas RLS (Supabase)

Todas las tablas de `casarural-v2` deben tener RLS habilitado. Las políticas deben basarse en `property_users` como tabla de membresía central.

5.1 Políticas de Lectura (casarural-v2)

```
-- Habilitar RLS en todas las tablas
ALTER TABLE properties      ENABLE ROW LEVEL SECURITY;
ALTER TABLE reservas        ENABLE ROW LEVEL SECURITY;
ALTER TABLE pagos            ENABLE ROW LEVEL SECURITY;
ALTER TABLE unidades         ENABLE ROW LEVEL SECURITY;
ALTER TABLE property_users   ENABLE ROW LEVEL SECURITY;
ALTER TABLE reservation_holds ENABLE ROW LEVEL SECURITY;
ALTER TABLE bloqueos         ENABLE ROW LEVEL SECURITY;

-- Usuario ve solo sus properties
CREATE POLICY "users_see_own_properties" ON properties
FOR SELECT USING (
  id IN (
    SELECT property_id FROM property_users
    WHERE user_id = auth.uid() AND active = true
  )
);

-- Usuario ve solo reservas de sus properties
CREATE POLICY "users_see_own_reservas" ON reservas
FOR SELECT USING (
  property_id IN (
    SELECT property_id FROM property_users
    WHERE user_id = auth.uid() AND active = true
  )
);

-- Patrón idéntico para: pagos, unidades, bloqueos, reservation_holds
CREATE POLICY "users_see_own_pagos" ON pagos
FOR SELECT USING (
  property_id IN (
    SELECT property_id FROM property_users
    WHERE user_id = auth.uid() AND active = true
  )
);

-- Usuario ve solo sus propias membresías
CREATE POLICY "users_see_own_memberships" ON property_users
FOR SELECT USING (user_id = auth.uid());
```

5.2 Políticas de Escritura (casarural-v2)

```
-- Insertar reservas solo en properties propias
CREATE POLICY "users_insert_reservas" ON reservas
FOR INSERT WITH CHECK (
  property_id IN (
    SELECT property_id FROM property_users
    WHERE user_id = auth.uid() AND active = true
  )
);

-- Actualizar reservas solo en properties propias
CREATE POLICY "users_update_reservas" ON reservas
FOR UPDATE USING (
  property_id IN (
    SELECT property_id FROM property_users
    WHERE user_id = auth.uid() AND active = true
  )
);
```

5.3 Regla sobre Service Role

Edge Functions con service_role bypassan RLS — requieren control explícito

Las Edge Functions de provisioning desde NexCore usan service_role para operar sobre SUPABASE-A. Esto es correcto y necesario. PERO estas funciones deben: (1) validar su propio secret/token de autenticación entre proyectos, (2) validar que el client_id y property_id existen y son coherentes, (3) escribir audit_log de cada operación ejecutada.

6. Edge Functions Necesarias

6.1 NexCore — SUPABASE-B

Función	Input	Output	Lógica crítica
create-client	datos comerciales	client_id	validar email único, audit_log
update-client-status	client_id, status	ok	suspender = sync a casarural-v2
create-subscription	client_id, plan_id, cycle	subscription_id	crear Stripe Customer + Subscription
change-plan	client_id, new_plan_id	ok	proration en Stripe + sync límites
cancel-subscription	client_id	ok	cancel en Stripe, no borrar datos
provision-casarural-instance	client_id, plan_id	instance_id	crear user Auth en SUPABASE-A, property, sync config
sync-client-config	client_id	ok	push plan_code, max_units, flags a casarural-v2
suspend-client-access	client_id	ok	subscription_status=SUSPENDED → sync
stripe-billing-webhook	Stripe event	200	invoice.paid, payment_failed, sub.updated
get-client-status	client_id	summary completo	plan, suscripción, instancias, Stripe Connect

6.2 casarural-v2 — SUPABASE-A (Bloque Stripe Connect)

Función	Propósito
receive-platform-config-sync	Recibe sync firmado de NexCore. Actualiza plan_code, max_units, flags, subscription_status en properties.
create-connected-account	Crea Stripe Connect Express account para una property. Idempotente.
create-connect-onboarding-link	Genera URL de onboarding alojado de Stripe. No persiste el link.
get-connect-account-status	Consulta estado real de la cuenta en Stripe. Actualiza charges_enabled, payouts_enabled, etc.
create-dashboard-login-link	Genera login link de un solo uso para el Express Dashboard. Solo cuando soporte lo indique.
create-stripe-checkout	Crea sesión de Checkout en direct charges. Recalcula precio en backend. Nunca confía en el frontend.
stripe-webhook-connect	Procesa eventos Connect. Confirma reservas por webhook, no por success_url. Idempotente por event.id.
refund-reservation	Reembolso total o parcial. Ejecuta en la connected account correcta. Nuevo registro en pagos.
check-availability	Excluye: reservas confirmadas + bloqueos manuales + iCal + holds activos no expirados.
calculate-price	Calcula precio en backend. Jerarquía: especial → base → dinámico → límites min/max.

7. Flujo Completo de Alta de Cliente

Fase 0 — NexCore (pre-provisioning)

```
Staff crea cliente en NexCore
→ saas_clients INSERT (status = ACTIVE)
→ saas_subscriptions INSERT (status = TRIAL | ACTIVE)
→ Stripe: crear Customer → stripe_customer_id
→ Stripe: crear Subscription → stripe_subscription_id
→ audit_logs INSERT
→ provisioning_jobs INSERT (action = CREATE INSTANCE, status = PENDING)
```

⚠ Qué puede fallar: Stripe rechaza la creación (datos incompletos, cuenta no verificada). Estado queda PENDING. El job tiene retry con backoff.

Fase 1 — Provisioning Técnico en casarural-v2

```
Edge Function: provision-casarural-instance (desde NexCore → SUPABASE-A)

PASO 1: crear usuario en Supabase Auth (SUPABASE-A)
→ Admin API de Supabase Auth
→ enviar magic link o password temporal
→ guardar supabase_user_id en client_product_instances
[CHECKPOINT → steps.step1 = done]

PASO 2: crear property base en casarural-v2
→ INSERT en properties con datos minimos
→ INSERT en property_users (user ↔ property)
→ marcar first_login = true
[CHECKPOINT → steps.step2 = done]

PASO 3: sync configuración
→ plan_code, max_units, debacu_enabled, subscription_status
→ platform_config_version = 1
[CHECKPOINT → steps.step3 = done]

PASO 4: actualizar provisioning_jobs (status = DONE)
PASO 5: actualizar client_product_instances (status = ACTIVE, onboarding_status = PENDING)
```

⚠ Qué puede fallar: SUPABASE-A no responde, error de Auth API. El job queda en FAILED con error_message. Al reintentar, los CHECKPOINTS evitan recrear lo que ya existe.

Fase 2 — Primer Login del Cliente (en casarural-v2)

```
Cliente recibe magic link / password
→ login con Supabase Auth (SUPABASE-A)
→ detectar first_login = true (flag en properties)
→ redirigir al wizard de configuración
```

Fase 3 — Wizard de Configuración (6 Pasos)

#	Nombre	Qué ocurre	Edge Function
1	Datos legales	Nombre comercial, NIF, email, dirección, tipo negocio. Guardar en properties.	— (directo a BD)
2	Crear/completar property	INSERT/UPDATE en properties con datos legales y comerciales completos.	— (directo a BD)
3	Crear cuenta Connect	Crear Stripe Connect Express. Guardar stripe_account_id. Idempotente.	create-connected-account

4	Generar onboarding link	Crear account_link con return_url y refresh_url. URL efímera, no persistir.	create-connect-onboarding-link
5	Retorno desde Stripe	Consultar estado real de la cuenta. Actualizar charges_enabled, payouts_enabled. Decidir si puede seguir.	get-connect-account-status
6	Configurar negocio	Textos, fotos, unidades, temporadas, políticas, branding. El wizard puede continuar aunque charges_enabled sea false.	— (módulos separados)

REGLA — El wizard debe ser recuperable

Si el usuario abandona entre pasos, al volver el wizard debe detectar en qué paso quedó basándose en los flags de la property: stripe_account_id null = Paso 3 pendiente; charges_enabled false = Paso 5 pendiente. Nunca reiniciar el wizard desde cero.

8. Detección de Problemas — Análisis Crítico

Los siguientes problemas se han identificado al cruzar ambos documentos fuente con los requisitos de producción. Deben resolverse antes o durante la implementación, no después.

8.1 Problema: Mecanismo de Sync NexCore → casarural-v2 No Está Definido

CRÍTICO — Dependencia sin implementación concreta

Los documentos mencionan que NexCore "empuja configuración" pero no existe ningún mecanismo concreto definido. ¿HTTP directo? ¿Cola de eventos? Sin esto, casarural-v2 opera con configuración desactualizada.

Solución requerida:

- Definir Edge Function receive-platform-config-sync en SUPABASE-A
- Autenticar con un secret compartido entre proyectos Supabase (variable de entorno NEXCORE_SYNC_SECRET)
- NexCore llama a esta función tras cada cambio de plan, suspensión o cambio de flags
- Usar platform_config_version para detectar configuraciones desactualizadas

8.2 Problema: Sin Webhook Inverso casarural-v2 → NexCore

ALTO — Estado Stripe Connect no se propaga a NexCore

Cuando el cliente completa el onboarding de Stripe Connect en casarural-v2, NexCore no se entera. El campo charges_enabled queda desincronizado en NexCore.

Solución requerida:

- Al finalizar get-connect-account-status en casarural-v2, hacer callback HTTP a NexCore
- NexCore actualiza client_stripe_accounts con el nuevo estado

8.3 Problema: Idempotencia de Provisioning Sin Garantía Formal

ALTO — Jobs que fallan a mitad dejan estado inconsistente

Si provision-casarural-instance falla entre el Paso 1 (user Auth creado) y el Paso 2 (property no creada aún), el reintento puede crear un segundo user Auth o fallar al intentar crear la property de nuevo.

Solución requerida: tabla provisioning_jobs con campo steps (jsonb) que guarda el estado de cada paso atómico.

```
-- Ejemplo de campo steps en provisioning_jobs
{
  "step1_auth_user": "done",      -- no volver a ejecutar
  "step2_property": "failed",    -- reintentar desde aquí
  "step3_sync_config": "pending"
}
```

8.4 Problema: Vinculación de Usuarios Entre Proyectos Supabase

ALTO — No está definido cómo pasar credenciales al nuevo cliente

El provisioning crea el usuario en SUPABASE-A (Auth), pero los documentos no especifican si se

envía magic link, password temporal, o invitación. Tampoco cómo se guarda el `supabase_user_id` en NexCore para referencias cruzadas.

Solución requerida:

- Usar Supabase Admin API para crear usuario en SUPABASE-A con `inviteUserByEmail` o `createUser`
- Guardar el UUID resultante en `client_product_instances.supabase_user_id`
- Enviar email de bienvenida con magic link al cliente

8.5 Problema: Naming Ambiguo en Campos Stripe

MEDIO — Riesgo de confusión entre Stripe Billing y Stripe Connect

Ambos contextos usan `stripe_customer_id`, `stripe_subscription_id`. Un desarrollador puede consultar un ID de SaaS en el contexto Connect o viceversa.

Solución: Prefijo explícito en NexCore:

- `saas_stripe_customer_id` (no `stripe_customer_id`)
- `saas_stripe_subscription_id` (no `stripe_subscription_id`)

8.6 Problema: Política de Suspensión Sin Niveles Definidos

MEDIO — Cortar acceso sin criterio puede generar reclamaciones

Si hay reservas pagadas en curso y se suspende el acceso total, el propietario no puede gestionar esas reservas. Falta definir qué funcionalidades se bloquean en cada tipo de suspensión.

Solución: Tres niveles de suspensión:

Estado	Restricciones
PAYMENT_ISSUE	Acceso lectura OK. No nuevas reservas. No nuevas unidades. Stripe Connect sin cambios.
TRIAL_EXPIRED	Acceso lectura solo. No acción. Stripe Connect bloqueado para nuevas sesiones.
POLICY_VIOLATION	Acceso bloqueado total. Solo datos de contacto. Stripe Connect suspendido.

8.7 Problema: Debacu No Puede Implementarse Como Módulo Interno

CRÍTICO — Si se implementa Paso 17 antes del Paso 18, habrá deuda técnica irreversible

El Paso 17 (inteligencia operativa) tienta a meter lógica de Debacu dentro de casarural-v2. Si se hace así, luego habrá que dismantelar auth, permisos, facturación y endpoints para convertirlo en API independiente.

Regla absoluta: Desde el primer día, las llamadas a Debacu desde casarural-v2 deben ir vía HTTP a una URL configurable en variables de entorno (`DEBACU_API_URL`). Nunca como lógica interna.

9. Propuesta de Mejoras

9.1 Mejoras Estructurales Prioritarias

Mejora 1 — Protocolo de Sync Entre Proyectos

Definir y documentar explícitamente la autenticación entre SUPABASE-B y SUPABASE-A:

```
// En SUPABASE-A: variables de entorno
NEXCORE_SYNC_SECRET=<secret_compartido>

// Edge Function: receive-platform-config-sync
// Valida: Authorization: Bearer <NEXCORE_SYNC_SECRET>
// Input: { client_id, plan_code, max_units, debacu_enabled, subscription_status, config_version }
// Acción: UPDATE properties SET plan_code=..., max_units=..., platform_config_version=...
// Output: { ok, applied_at, property_id }
```

Mejora 2 — Provisioning con Pasos Atómicos

El campo steps en provisioning_jobs permite retry quirúrgico:

```
// Al iniciar el job
steps = { step1: 'pending', step2: 'pending', step3: 'pending', step4: 'pending' }

// Tras cada paso exitoso, ANTES de continuar:
UPDATE provisioning_jobs SET steps = jsonb_set(steps, '{step1}', '"done"') WHERE id = job_id

// Al reintentar: omitir los steps marcados como 'done'
```

Mejora 3 — platform_config_version para Detectar Configs Obsoletas

Cada sync desde NexCore incrementa platform_config_version. casarural-v2 puede comparar su versión con la esperada y saber si necesita re-sync sin tener que consultar NexCore en cada request.

9.2 Cambios de Naming Recomendados

Nombre en documentos	Nombre recomendado	Motivo
platform_clients	saas_clients	Contexto B2B SaaS más claro
platform_plans	saas_plans	Consistencia con saas_clients
platform_apps	client_product_instances	Más descriptivo y no ambiguo
platform_payment_accounts	client_stripe_accounts	Evita confusión con pagos de reservas
platform_provisioning_jobs	provisioning_jobs	Simplificar, el prefijo es redundante
stripe_customer_id (NexCore)	saas_stripe_customer_id	Distingue de Stripe Connect

9.3 Separación de Responsabilidades — Regla de Oro

Quién	Decide	Ejecuta	Muestra
NexCore	✓ Planes, límites, features, estado	✓ Provisioning, sync	Panel staff interno

casarural-v2	x No inventa reglas de plan	✓ Reservas, pagos, Connect	Panel del propietario
Frontend	x Nunca lógica de negocio	x Solo consume APIs	✓ UI al usuario
Debacu	✓ Score de riesgo	✓ Evaluate, incidents	API → terceros

10. Prioridad de Implementación

Fase 1 — Base SaaS (NexCore mínimo viable)

Objetivo: poder dar de alta clientes manualmente y provisionar casarural-v2.

#	Tarea	Entregable
1	Tablas NexCore: <code>saas_clients</code> , <code>saas_plans</code> (con datos), <code>saas_subscriptions</code> , <code>client_product_instances</code> , <code>provisioning_jobs</code> , <code>audit_logs</code>	Schema SQL completo
2	Edge Functions: <code>create-client</code> , <code>create-subscription</code> , <code>provision-casarural-instance</code>	3 Edge Functions
3	Edge Function: <code>receive-platform-config-sync</code> en SUPABASE-A (con autenticación por secret)	1 Edge Function + secret
4	Edge Function: <code>sync-client-config</code> en NexCore	1 Edge Function
5	Panel admin mínimo: lista clientes + alta + estado provisioning	UI básica

Fase 2 — Stripe Billing SaaS

Objetivo: cobrar cuotas SaaS automáticamente y gestionar suspensiones.

- Integrar Stripe Billing en `create-subscription`
- Edge Function `stripe-billing-webhook` en NexCore
- Suspensión automática por impago + sync a casarural-v2
- Tabla `saas_invoices` y visualización de facturas en panel

Fase 3 — Stripe Connect (Cobro de Reservas)

Objetivo: que el cliente pueda cobrar reservas de huéspedes. Implementar en este orden exacto:

1. `create-connected-account` — base de todo lo demás
2. `create-connect-onboarding-link` — para el wizard
3. `get-connect-account-status` — verdad real tras onboarding
4. `create-dashboard-login-link` — acceso asistido solo cuando soporte lo indique
5. `reservation_holds` + anti-overbooking — ANTES de cobrar en producción
6. `create-stripe-checkout` — direct charges sobre la connected account
7. `stripe-webhook-connect` — confirmar reservas solo desde webhook, nunca desde `success_url`
8. `refund-reservation` — total y parcial sobre la connected account correcta

Fase 4 — Integraciones y Capa Diferencial

- Emails transaccionales con Resend (confirmación, refund, cancelación, pre-checkin, checkout, post-estancia)
- iCal import/export por unidad — previene overbooking con OTAs
- Debacu API pública independiente — endpoints: POST /v1/evaluate, POST /v1/incidents, GET /v1/status
- casarural-v2 consume Debacu vía HTTP — snapshot de riesgo en reservation_risk_snapshots

Fase 5 — Operativa Avanzada y Reporting

- Panel de reservas profesional con ficha completa y acciones operativas
- Backoffice de incidencias — tabla operational_incidents con detección automática
- CRM / pre-reservas / presupuestos (revisar plantillas de email existentes)
- Dashboard financiero — ingresos netos (pagos - refunds), ocupación, ADR, RevPAR
- Pricing dinámico — ajuste por ocupación dentro de límites min/max definidos por el cliente
- Automatizaciones — recordatorios, post-estancia, limpieza, expiración de holds y pre-reservas
- Capa de inteligencia — tabla operational_insights, insights de ocupación, pricing y riesgo

Decisiones Críticas a Cerrar Antes de Escribir Código

Decisión 1

Protocolo de sync NexCore → casarural-v2: definir el mecanismo exacto (HTTP firmado, secret compartido, función receive-platform-config-sync). Sin esto, los límites de plan no se propagan.

Decisión 2

Estrategia de provisioning resiliente: máquina de estados con pasos atómicos en el campo steps de provisioning_jobs. Sin esto, un fallo parcial deja el sistema en estado inconsistente.

Decisión 3

Debacu como API HTTP desde el primer día: nunca como módulo interno de casarural-v2. La URL de Debacu debe ser una variable de entorno (DEBACU_API_URL) en SUPABASE-A.

Decisiones:

Te doy contestación cerrada a las 3 últimas cuestiones del documento, que son estas: definir el sync NexCore → casarural-v2, cerrar el provisioning resiliente y decidir ya que Debacu va como API separada desde el primer día. Esas tres aparecen literalmente como decisiones críticas antes de escribir código.

1) Protocolo de sync NexCore → casarural-v2

Mi respuesta es: **hazlo por HTTP firmado, con Edge Function receptora en SUPABASE-A, secret compartido y versión de configuración**. No lo dejes “implícito”, porque ahí es donde luego empiezan los desfases de plan, límites y flags. El propio análisis ya apunta a ese camino con receive-platform-config-sync, NEXCORE_SYNC_SECRET y platform_config_version.

La decisión correcta sería esta:

- **NexCore manda siempre**
- **casarural-v2 nunca decide el plan**

- casarural-v2 solo guarda una **copia operativa mínima**
- cada cambio de plan, suspensión, activación de Debacu o cambio de límites dispara un sync desde Core

Contrato práctico recomendable:

POST /functions/v1/receive-platform-config-sync

Body mínimo:

```
{
  "client_id": "uuid-core",
  "external_property_id": "uuid-a",
  "plan_code": "PRO",
  "max_units": 3,
  "max_domains": 1,
  "max_users": 2,
  "debacu_enabled": true,
  "subscription_status": "ACTIVE",
  "platform_config_version": 7,
  "sent_at": "2026-04-12T15:30:00Z"
}
```

Cabecera:

Authorization: Bearer <NEXCORE_SYNC_SECRET>

X-Sync-Version: 7

Reglas que dejaría fijadas:

- si platform_config_version recibido es **menor** que el guardado, se ignora
- si es **igual**, la operación debe ser idempotente
- si es **mayor**, se aplica y se actualiza platform_last_synced_at
- cada sync debe quedar en audit log en ambos lados
- no metas 50 campos; solo límites, flags y estado de suscripción, como ya se sugiere en el preanálisis.

Mi recomendación realista: **no montes cola/event bus ahora**. Para empezar, HTTP firmado entre Edge Functions es suficiente y mucho más rápido de cerrar. Ya más adelante, si el volumen crece, lo conviertes en eventos. El preanálisis ya dice justo eso: Edge Functions directas valen para arrancar.

2) Estrategia de provisioning resiliente

Mi respuesta es: **sí o sí con máquina de estados y pasos atómicos en**

provisioning_jobs.steps. No lo dejes en una sola función “larga” que hace todo seguido. Si falla a mitad, se te queda el cliente en tierra de nadie. El análisis lo marca como problema alto y propone exactamente el campo steps para retry quirúrgico.

Yo lo dejaría así:

Estados del job:

- PENDING
- RUNNING
- FAILED
- DONE

Pasos internos:

```
{
  "create_auth_user": "pending",
  "create_property": "pending",
  "link_property_user": "pending",
  "sync_platform_config": "pending",
}
```

```

    "send_welcome_email": "pending"
  }
  Y cada paso con metadatos:
  {
    "create_auth_user": {
      "status": "done",
      "at": "2026-04-12T15:45:00Z",
      "external_user_id": "uuid-auth-a"
    }
  }
}

```

Reglas operativas:

- antes de ejecutar un paso, lees steps
- si ya está done, no lo repites
- tras cada paso correcto, actualizas steps **antes** de pasar al siguiente
- si falla, guardas error_message, incrementas attempts y sales
- el reintento continúa desde el primer paso no completado
- pon límite de reintentos, por ejemplo 5
- si falla algo externo, como crear usuario en Auth o escribir en SUPABASE-A, no intentes “adivinar”; deja el job en FAILED y reintenta desde estado

Orden correcto, sin adornos:

1. crear cliente y suscripción en NexCore
2. crear provisioning_job
3. crear usuario Auth en SUPABASE-A
4. crear property base en SUPABASE-A
5. crear relación en property_users
6. sincronizar límites y flags
7. marcar onboarding pendiente
8. enviar acceso inicial

Y algo importante: **cada paso debe ser idempotente por clave de negocio**, no solo por estado del job. Por ejemplo:

- usuario Auth: por email
- property provisionada: por platform_client_id
- instancia producto: por (client_id, product_code)

Eso también está alineado con el índice único de client_product_instances que sale en el análisis.

Mi respuesta corta aquí es: **sin steps atómicos no empieces el provisioning**. Porque luego arreglarlo cuesta bastante más que hacerlo bien ahora.

3) Debacu como API HTTP desde el primer día

Mi respuesta es clara: **sí, completamente separada desde el día 1**. No la metas dentro de casarural-v2 ni “provisionalmente”. El documento lo deja bastante claro: si haces eso, luego tendrás deuda técnica seria y tocará dismantelar auth, permisos, endpoints y facturación.

La arquitectura correcta es esta:

- **NexCore** decide si Debacu está permitido por plan
- **casarural-v2** solo muestra o consume Debacu si debacu_enabled = true
- **Debacu** vive en SUPABASE-C con Edge Functions propias y auth propia
- la comunicación entre casarural-v2 y Debacu va por **HTTP backend-to-backend**

- la URL debe ser variable de entorno, por ejemplo DEBACU_API_URL, como se indica en el análisis.

Endpoints mínimos para arrancar, que ya aparecen sugeridos:

- POST /v1/evaluate
- POST /v1/incidents
- POST /v1/manual-check
- GET /v1/status

Yo añadiría estas reglas:

- auth por client_id + api_key hash en BD
- rate limit simple por cliente
- nunca exponer PII de terceros
- identity normalization/HMAC desde backend
- casarural-v2 no guarda "la verdad" del riesgo; solo un snapshot útil para la reserva o consulta
- si Debacu cae, casarural-v2 no debe romper la reserva: debe degradar con estado risk_unavailable

Eso último importa mucho. Debacu debe aportar valor, pero no convertirse en un punto único de caída para vender una reserva.

Mi cierre práctico

Las tres decisiones, ya cerradas como yo las dejaría, son estas:

Decisión 1

HTTP firmado entre proyectos, con receive-platform-config-sync, NEXCORE_SYNC_SECRET y platform_config_version. NexCore decide; casarural-v2 aplica.

Decisión 2

Provisioning con provisioning_jobs + steps atómicos + retry idempotente. Sin eso, acabarás con clientes medio creados y soporte manual.

Decisión 3

Debacu separada como API HTTP desde el principio, consumida por casarural-v2 vía DEBACU_API_URL, nunca embebida dentro del producto operativo.

La respuesta sensata es que **las tres hay que aceptarlas tal cual y convertirlas ya en decisión de arquitectura**, porque ninguna de las tres es decorativa. Las tres afectan al núcleo del sistema y si las dejas abiertas, luego el proyecto se vuelve inconsistente.